

WCMS

CSP Style 設計與開發指南

給 UI / UX 設計與協作同仁的學習版

平台 Runtime -> Balanced

React Component -> Strict-Compatible

CMS Content -> Balanced + Sanitizer

版本：2026-08-12 | 依據：Feature/Dev 正式規範與現行程式案例

一句話先記住

WCMS 的「Balanced」是平台對 Content 相容性的 Runtime 選擇；不是讓 React Component 使用 Inline Style 的許可。React UI 一律按 Strict 強度設計與實作。

1. 先理解：Balanced / Strict 是什麼？

這兩個名稱是 WCMS 專案自己的 Style CSP 模式，不是 W3C CSP 規格的官方分級名稱。

CSP (Content Security Policy) 是瀏覽器的內容安全政策。對 UI Style 來說，最重要的兩個概念是：外部／樣式區塊，以及 HTML 元素上的 style Attribute。

控制面	W3C CSP 指令	直覺理解
樣式表 / <style>	style-src / style-src-elem	哪些樣式來源或樣式區塊可以執行
style="..."	style-src-attr	元素上的 Inline Style 是否允許

WCMS 三種模式的差異

模式	style-src	style-src-attr	定位
legacy	允許 unsafe-inline	允許 unsafe-inline	舊相容模式，最寬鬆
balanced	不開放一般 inline <style>	允許 unsafe-inline	保留 Content style Attribute 彈性
strict	不開放一般 inline <style>	none	不允許 style Attribute，最嚴格

重點

Balanced 與 Strict 的核心差異，在 WCMS 目前設計中主要集中在 style-src-attr：Balanced 允許 style="..."，Strict 完全禁止。Script CSP 並沒有因 Balanced 而放寬。

W3C CSP Level 3 明確區分 style-src-elem 與 style-src-attr：前者管理非 inline attribute 的樣式，後者專門管理 style Attribute。

2. WCMS 平台政策：Runtime Balanced，程式碼 Strict-Compatible

平台將綁定 Balanced 強度，是為了讓 CMS Content 保留必要的排版彈性；但 WCMS 自行開發的 React UI 不跟著放寬。這是刻意把「執行環境」和「開發規範」分開。

平台 Runtime Balanced	產品程式碼 Strict-Compatible	CMS Content Balanced 例外
瀏覽器實際收到的 CSP 保留 Content style Attribute	React / TSX / 共用 UI 禁止新增 Inline Style	TinyMCE / 舊 Content 允許有限度 style Attribute

為什麼不直接全站 Strict？

- React Component 是我們可以控制與重構的程式碼，可以逐步做到 Strict-Compatible。
- CMS Content 是使用者建立的 HTML，常見編輯器會產生 text-align、width、color 等 style Attribute；全站 Strict 會讓既有內容大量失去排版。
- 因此 WCMS 保留 Content 的相容性，但不把這個彈性擴散到產品程式碼。

設計原則

Platform Balanced = 產品相容性策略；Component Strict = 工程品質與未來安全遷移策略。兩者不是矛盾，而是責任分層。

注意：此文件描述 2026-08-12 已定義的設計政策。若部署環境仍在參數切換期間，實際 Header 仍以該環境設定為準。

3. React Component：UI Style 一律以 Strict 強度設計

UI/UX 在設計交付時，請優先把畫面表達成「狀態、Variant、Token、Class」；RD 實作時，不應依賴 style Attribute 才能成立。

寫法	Strict-Compatible	UI/UX / RD 建議
className + 正式 CSS	是	首選。固定排版、顏色、間距、狀態都走 Class
CSS Module	是	適合 Component 封裝
data-* + CSS Selector	是	適合狀態切換、尺寸級距、Variant
Bootstrap / 共用 Utility Class	是	已有共用能力時直接沿用
React style={{...}}	否	新 Code 禁止作為一般 UI Style
setAttribute("style", ...)	否	禁止
style.cssText = ...	否	禁止
element.style.xxx = ...	規範上禁止一般使用	CSSOM 實際阻擋需 Runtime 判斷，但開發規範仍不採用
Feature/Spec 自建 <style>	否	改成正式 CSS 或共用動態樣式機制

範例：固定樣式

NG - React Inline Style

```
<div style={{ display: "inline-flex", marginTop: "16px" }} />
```

OK - Class + CSS

```
<div className="qr-link qr-link--spaced" />
```

```
.qr-link { display: inline-flex; }
.qr-link--spaced { margin-top: 1rem; }
```

範例：動態狀態

OK - 狀態用 Class

```
<div className={isActive ? "card card--active" : "card"} />
```

UI/UX 交付建議：如果設計需要 3 種尺寸、4 種間距、2 種狀態，請直接定義成 Variant / Token；不要把需求描述成「這裡讓前端算任意 px 再塞 style」。

4. CMS Content：有限度使用 Balanced 允許的 Style

Content 是 WCMS 特別保留彈性的區域。Balanced 允許 style Attribute，主要是為了 TinyMCE、舊資料與使用者自定義排版相容。這是「受控例外」，不是任意 HTML/CSS 的自由區。

Content 寫法	Balanced	Strict	建議
<p class="...">	可	可	優先
<p style="text-align:center">	可	不可	Content 必要時可接受
	可	不可	可用，但要兼顧 RWD / AA
<style>...</style>	不可（目前設計）	不可	不要讓 Content 自帶整包 CSS
<script> / onclick=	不可	不可	Sanitizer + Script CSP 都應阻擋

已知風險

- 如果系統另外存在 HTML Injection，Balanced 允許的 style Attribute 可能被拿來遮蔽內容、覆蓋畫面或做視覺欺騙。
- 惡意或不當 Style 也可能破壞 AA：例如隱藏焦點、把內容移出畫面、用固定定位蓋住主要資訊。
- 因此 Content 仍必須經過 CMS Sanitizer、URL / HTML 安全檢查與權限控制；Balanced 不是信任 Content。
- 弱掃工具可能把 unsafe-inline Style Attribute 視為較弱 CSP；是否形成 Finding 與嚴重度，仍以專案實際掃描 Policy 為準。

對 UI/UX 的界線

可以設計「文字置中、字級、色彩、表格寬度」等 Content 編輯能力；但不建議把「任意 CSS 編輯器」當成一般使用者功能。

建議 Content 能力優先順序

- 第一層：編輯器既有格式工具（對齊、字級、粗體、表格、圖片尺寸）。
- 第二層：WCMS 預設 Style / Class 選單。
- 第三層：真正客製 CSS 僅保留給進階角色，且需另做 Scope / Sanitization 設計。

5. AI 白箱檢測：現在會專門檢查 Strict-Compatible UI Style

Feature/Dev 的前端開發規範已新增「5.7 UI Style 必須符合 Strict CSP 開發強度」；需求與單元白箱檢測也新增「5.4 React UI Style Strict-Compatible 檢測」。

AI 必查項目

- React style={{ ... }}
- WCMS 自行產生 style="..."
- setAttribute("style", ...) / style.cssText
- element.style.xxx 作為一般 UI 樣式機制
- Feature / Spec 未受管理的 <style> 或動態 Style Injection
- 能用 className / data-* / CSS 解決卻仍使用 Inline Style
- 第三方 / Legacy 是否被隔離在最小 Wrapper / Adapter，並留下原因與移除條件

白箱輸出範例

AI 白箱 Finding 範例

【規範定位】前端開發規範 5.7

【結果】不符合

【程式證據】<LangNavLink style={{ display: "inline-flex" }}>

【影響】React Component 不符合 Strict-Compatible 規範

【建議方向】移至正式 CSS Class

【需要的驗證】Typecheck / Build；若涉及 CSSOM 再做 Runtime CSP 驗證

【RD 判定】待 RD 判定

重要

AI 不會因為平台 Runtime 是 Balanced 就放寬 React UI 的判定；相反地，Content style Attribute 不套 Component Strict 規則，改由 Content 安全流程檢查。

AI 白箱是靜態規範檢查，不等同 AppScan、Freego、瀏覽器實機或 QA。AI 不應自行宣稱弱掃或 Runtime 已通過。

6. Feature/Dev 實際案例：發生什麼、怎麼改？

以下案例均取自 Feature/Dev 現行程式。目的是讓 UI/UX 理解哪些設計／實作型態在 Strict 下會變成相容性問題。

案例	目前寫法	Strict 風險	建議方向
Grid 欄寬 / Resize	th style={{ width, position }}	欄寬與拖曳控制可能失效	固定樣式移 CSS；動態寬度走共用 Strict-Compatible 機制
Action Overlay	大量 style={{...}} + inline <style>	Overlay、Spinner 樣式整體失效	完整移到 ActionOverlay.css
Banner	aspectRatio、transition、CSS variable、圖片尺寸皆 inline	輪播比例、轉場、圖片顯示可能異常	固定項目 CSS 化；動態值改 Token / 共用機制
QR Code Link	style={{ display:"inline-flex" }}	很小但明確不相容	改成 className

從 UI/UX 角度看

Static Style 幾乎都能直接改 CSS；真正需要討論的是「任意動態數值」的互動設計，例如 Grid 可拖曳欄寬、任意 Banner 比例、Runtime 轉場速度。這些需要 UI/UX 與 RD 一起決定是否改成有限 Token，或由平台提供正式動態樣式底層。

6.1 案例一：Grid 動態欄寬

Feature/Dev 的 SysCore Grid 目前直接在 <th> 上輸出寬度與定位，Resize Handle 也用 Inline Style。這是 Spec1821 正式環境曾出現欄寬跑掉的典型根因。

Feature/Dev - Grid_Comp.tsx (節錄)

```
<th
  style={{
    width: typeof col.width === "number" ? `${col.width}px` : "auto",
    position: "relative",
    color: "inherit"
  }}
>
```

Resize Handle (節錄)

```
<div
  style={{
    position: "absolute", right: 0, top: 0, bottom: 0,
    width: "5px", cursor: "col-resize", userSelect: "none"
  }}
/>
```

怎麼改

- position / color / handle 位置 / cursor 這些固定值直接移到 CSS Class。
- 「使用者任意拖曳的 px 欄寬」是真動態資料，不能只用一般固定 Class 解決。應由 SysCore 提供共用 Strict-Compatible Dynamic Style 機制，或產品設計改成有限欄寬 Token。
- 不要在每個 Feature 自己做一套 workaround，因為 Grid 是共用底層。

UI/UX 決策點

如果使用者一定要「任意拖曳到每個 px」，RD 需要平台級動態樣式方案；如果只需要「窄 / 中 / 寬」三種欄寬，則可以用 Token / Class 大幅降低複雜度。

6.2 案例二：Action Overlay 與 QR Code

ActionOverlay_Comp.tsx

這個 Component 的定位、背景、卡片、Spinner、文字粗細全部寫在 style={{...}}，甚至把 @keyframes 放進 inline <style>。在 Strict-Compatible 規範下，這屬於高密度的反例。

目前概念（節錄）

```
<div style={{ position:"fixed", inset:0, zIndex:4000, ... }}>
  <div style={{ background:"white", padding:"16px 20px", ... }}>
    <div style={{ width:22, height:22, animation:"spin 1s linear infinite" }} />
  </div>
  <style>{@keyframes spin { to { transform: rotate(360deg); } }}</style>
</div>
```

建議解法

改為正式 CSS

```
<div className="action-overlay">
  <div className="action-overlay__panel">
    <div className="action-overlay__spinner" aria-hidden="true" />
  </div>
</div>

/* ActionOverlay.css */
.action-overlay { position: fixed; inset: 0; ... }
.action-overlay__spinner { animation: spin 1s linear infinite; }
@keyframes spin { to { transform: rotate(360deg); } }
```

LibQRCode_Comp.tsx

QR Code Link 只有一個 display:inline-flex，也寫成 Inline Style。這種是最好修、也最應該在白箱直接抓出的案例。

建議

```
<LangNavLink className="qr-code-link"> ... </LangNavLink>

.qr-code-link { display: inline-flex; }
```

6.3 案例三：Banner 的 Dynamic Style

Banner_Comp.tsx 同時混合了「固定樣式」與「真正動態值」，這是 UI/UX 與 RD 最需要一起拆解的案例。

目前項目	類型	建議
img width / height / object-fit / min-height	固定 Style	直接移到 CSS Class
Overlay position / inset / z-index / pointer-events	固定 Style	直接移到 CSS Class
transitionDuration: speedMs	動態數值	若可接受，改成速度 Token；否則走共用動態樣式機制
aspectRatio: Width / Height	動態數值	優先使用預設比例 Token；必要時才走共用機制
--bs-carousel-transition-duration	動態 CSS variable	Inline CSS variable 仍不符合 Component Strict 規範，應改平台級處理

固定 Style 的解法

```
<img
  className="banner-image"
  src={imgUrl}
  alt={title}
/>

.banner-image {
  width:100%; height:100%; object-fit:cover; min-height:200px;
}
```

UI/UX 可協助的地方

把「可自訂速度 / 比例」定義成可理解的設計 Token，例如 slow / normal / fast、16:9 / 4:3 / 1:1，而不是預設允許任意數值。若業務真的需要任意值，再由 RD 提供共用 Strict-Compatible 技術方案。

7. UI / UX 協作 Checklist

以下 Checklist 可直接用在 Figma 規格、設計驗收或與 RD 討論時。

- ☐ 固定排版是否可以明確對應到 Class / Component Variant ?
- ☐ 顏色、間距、字級、圓角是否有 Token，而不是每頁任意值？
- ☐ 動態狀態是否能用 active / disabled / selected / size 等 Variant 描述？
- ☐ 是否真的需要任意 px / 任意比例 / 任意速度？還是可收斂成有限選項？
- ☐ 如果是 CMS Content，自定義能力是否屬於內容需求，而不是產品 Component 偷用 Inline Style？
- ☐ Content 的自定義樣式是否仍保留 AA / RWD / 可讀性？
- ☐ 設計變更是否會讓第三方 Carousel / Grid / Editor 需要大量 Runtime Style？需要時要提早找 RD 評估。
- ☐ PR / 交付前是否已讓 AI 白箱掃 React UI Style Strict-Compatible？

最終共識

WCMS Style Policy

產品 UI：用 Strict 的標準寫；平台 Runtime：用 Balanced 保留 Content 相容；Content：可以有限度使用 Balanced 允許的 Style，但必須經過 Sanitizer 與安全邊界。

參考來源 (2026-08-12)

- W3C Content Security Policy Level 3 - style-src, style-src-elem, style-src-attr。
- Feature/Dev: WCMS_Frontend/src/SSR/CSPSetting.ts。
- Feature/Dev: Docs/01_研發與技術文件/02_開發規範/02_前端開發規範.md，5.7 UI Style 必須符合 Strict CSP 開發強度。
- Feature/Dev: Docs/03_品質與驗證文件/01_AI 白箱檢測/01_需求與單元白箱檢測.md，5.4 React UI Style Strict-Compatible 檢測。
- Feature/Dev 實例: Grid_Comp.tsx、ActionOverlay_Comp.tsx、Banner_Comp.tsx、LibQRCode_Comp.tsx。
- 前端規範更新 Commit: a251ac576e247abc8d539d3bf7076be8950537ab。
- 白箱規範更新 Commit: d69562436b1275fed992c08228c6afd1ba31c97b。

備註：本文件是 UI/UX 學習與協作指南；安全掃描、AA、Browser Runtime 與正式 Release 驗證仍需使用各自工具與實際環境確認。